

Spark

Large-scale near-real-time stream processing

Hadoop Ecosystem

Data Visualization

SAS Visual Analytics

Tableau

Qlik

SAP Lumira

R

D3.JS

iCharts

Timeline JS

Apache Zeppelin

System Deployment

Apache Ambari

Apache Mesos

Marathon

Hortonworks HOYA

Apache Bigtop

Deploop

Apache Eagle

Cloudera HUE

Myriad

Brooklyn

Apache Helix

Buildoop

SequenceIQ Cloudbreak

Data Ingestion

Apache Flume

Apache Sqoop

Facebook Scribe

Apache Chukwa

Apache Kafka

Netflix Suro

Apache Samza

Cloudera Morphline

HIHO

Apache NiFi

Apache ManifoldCF

Service Programming

Apache Thrift

Apache Zookeeper

Apache Avro

Apache Curator

Apache Karaf

Twitter Elephant Bird

LinkedIn Norbert

Scheduling & DR

Apache Oozie

LinkedIn Azkaban

Apache Falcon

Shedoscope

Security

Apache Sentry

Apache Knox Gateway

Apache Ranger

Frameworks

Jumbune

Spring XD

Cask Data App Platform

Metadata

Metascope

Apache Tika

Machine Learning

Apache Mahout

WEKA

Cloudera Oryx

Deeplearning4j

MADlib

H2O

Sparkling Water

Apache SystemML

Distributed Programming

Apache Ignite

Apache MapReduce

Apache Pig

JAQL

Apache Spark

Apache Storm

Apache Flink

Apache Apex

Netflix PigPen

AMPLAB SIMR

Facebook Corona

Apache REEF

Apache Twill

Damballa Parkour

Apache Hama

Datasalt Pangool

Apache Tez

Apache DataFu

Kangaroo

TinkerPop

Pachyderm MapReduce

Apache Beam

SQL on Hadoop

Apache Hive

Apache HCatalog

Apache Trafodion

Apache HAWQ

Apache Drill

Cloudera Impala

Facebook Presto

Datasalt Splout SQL

Apache Tajo

Apache Phoenix

Apache MRQL

Kylin

NoSQL Databases

Wide Column

Apache HBase

Apache Cassandra

Hypertable

Apache Accumulo

Apache Kudu

Apache Parquet

Document

MongoDB

RethinkDB

ArangoDB

CouchDB

DynamoDB

Gemfire

Key-Value

Redis

LinkedIn Voldemort

RocksDB

OpenTSDB

Graph

Giraph

Neo4j

TitanDB

OrientDB

Stream Data Model

EventStore

NewSQL Databases

TokuDB

HandlerSocket

Akiban Server

Drizzle

Haeinsa

SenseiDB

Sky

BayesDB

InfluxDB

VoltDB

SAP HANA

Distributed File System

Apache HDFS

Red Hat GlusterFS

Quantcast File System

Ceph File System

Lustre File System

Alluxio

GridGain

XtreemFS

- **Speed** – Spark helps to run an application in Hadoop cluster, up to 100 times faster in memory, and 10 times faster when running on disk. This is possible by reducing number of read/write operations to disk. It stores the intermediate processing data in memory.
- **Supports multiple languages** – Spark provides built-in APIs in Java, Scala, or Python. Therefore, you can write applications in different languages. Spark comes up with 80 high-level operators for interactive querying.
- **Advanced Analytics** – Spark not only supports ‘Map’ and ‘reduce’. It also supports SQL queries, Streaming data, Machine learning (ML), and Graph algorithms.

What is Spark Streaming?

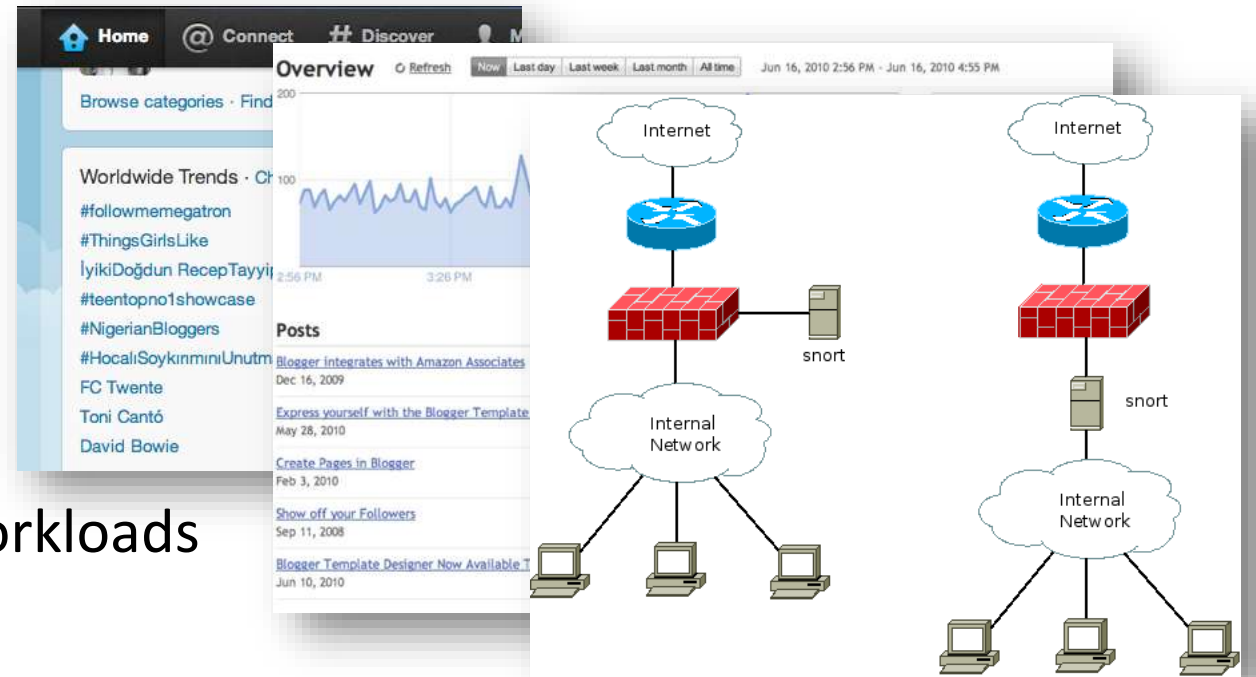
- Apache spark is an integrated, fast, in-memory, general purpose engine for large scale data processing.
- Framework for large scale stream processing
 - Scales to 100s of nodes
 - Achieves 10-100x performance over Hadoop by operating with an in-memory data construct called Resilient Distributed Datasets (RDDs)
 - Can achieve second scale latencies
 - Integrates with Spark's batch(iterative) and interactive processing
 - Spark offers built in libraries for Machine Learning, Graph Processing, Stream Processing and SQL to deliver superfast data processing along with high programmer productivity.
 - Spark is compatible with Hadoop file systems and tools.
 - Provides a simple batch-like API for implementing complex algorithm
 - Spark is an alternative to MapReduce rather than a replacement for Hadoop File System.
 - Can absorb live data streams from Kafka, Flume, ZeroMQ, etc.

More on Spark

- Apache Spark was developed in 2009 in UC Berkeley's AMPLab, and open sourced in 2010 as an Apache project.
- It can process data from a variety of data repositories including the HDFS, and NoSQL databases such as Hbase and Cassandra.
- Spark prioritizes in-memory processing to boost the performance of big data analytics applications, however, it can also do conventional disk based processing when data sets are too large to fit into available system memory.
- Spark enables applications in Hadoop clusters to run up to **100** times faster in memory and **10** times faster even when running on disk.

Motivation

- Many important applications must process large streams of live data and provide results in near-real-time
 - Social network trends
 - Website statistics
 - Intrusion detection systems
 - etc.
- Require large clusters to handle workloads
- Require latencies of few seconds



Need for a framework ...

... for building such complex stream processing applications

But what are the requirements
from such a framework?

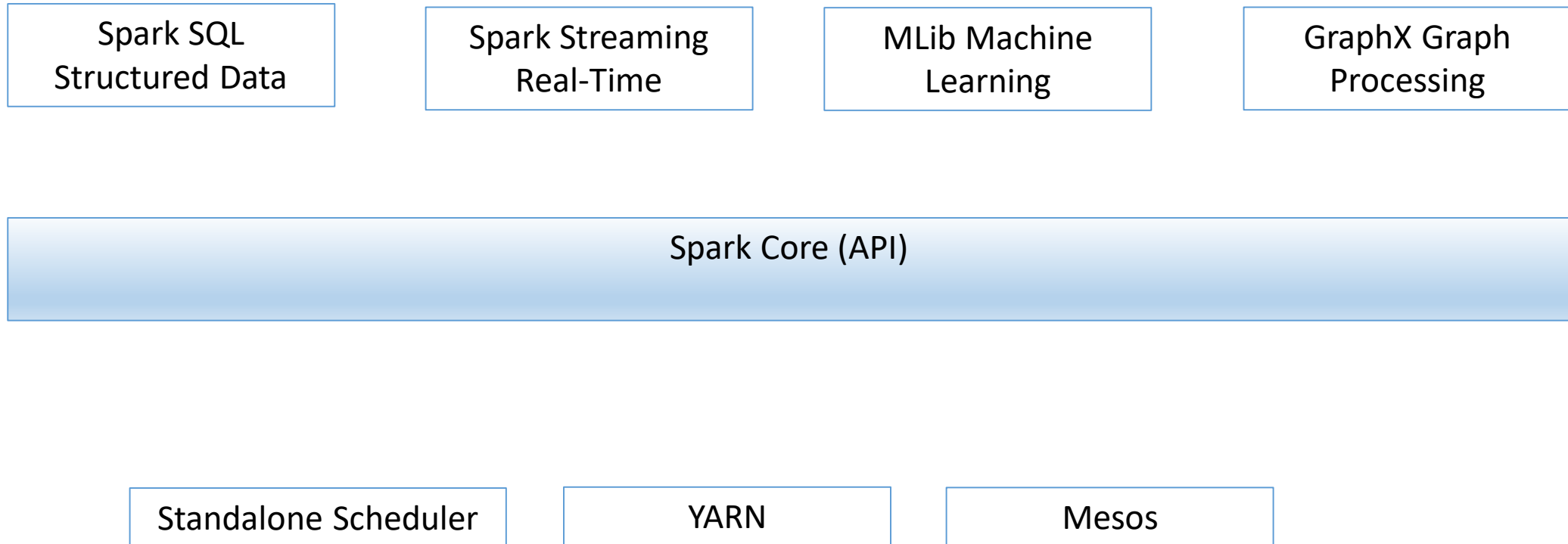
Requirements

- **Scalable** to large clusters
- **Second-scale** latencies
- **Simple** programming model

Spark Architecture

- The core spark engine functions partly as an API layer and underpins a set of related tools for managing and analyzing data.
- These includes
 - SQL query engine
 - A library for machine learning algorithms
 - A Graph processing system, and
 - Streaming data processing software
- Spark allows programmers to develop complex, multi-step data pipelines using DAG pattern. It also supports in-memory data sharing across DAGs, so that different jobs can work with the same data.
- Spark runs on top of existing HDFS infrastructure to provide enhanced and additional functionality.

SPARK Architecture



Resilient Distributed Datasets (RDDs)

- Allow apps to keep working sets in memory for efficient reuse
- Retain the attractive properties of MapReduce
 - Fault tolerance, data locality, scalability
- Support a wide range of applications
- RDDs is a distributed memory construct, these are motivated by two types of applications that current computing frameworks handle inefficiently.
 - Iterative algorithms
 - Interactive data mining tools
- In both the cases, keeping data in memory can improve performance by an order of magnitude.
- RDDs are immutable and partitioned collection of records.
 - They can only be created by coarse grained operations such as map, filter, group by, etc.
 - Coarse grain operations means that the operations are applied on all elements in a dataset.
- RDDs can only be created by:
 - Reading data from a stable storage such as HDFS or
 - Transformations on existing RDDs.

Resilient Distributed Datasets (RDDs)

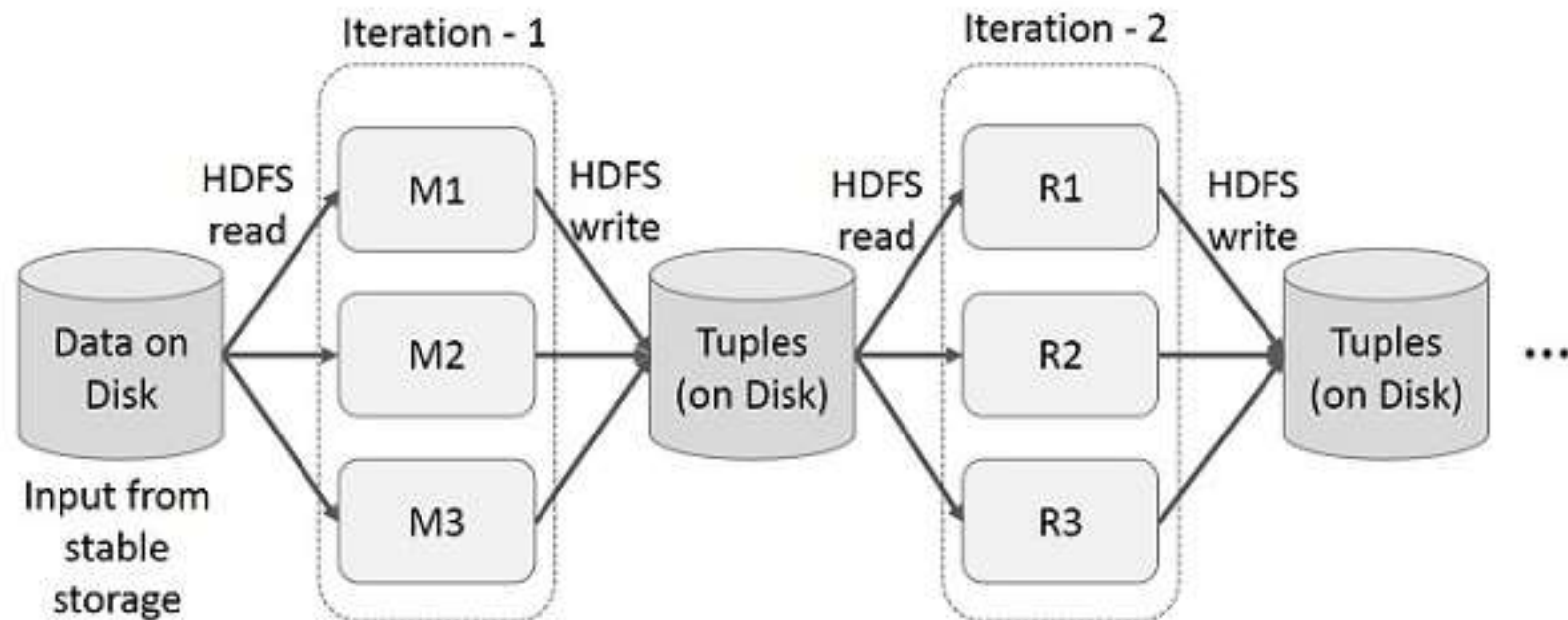
- Once data is read into an RDD object in Spark, a variety of operations can be performed on the RDD by invoking abstract spark APIs.
- The two major type of operation available are transformations and actions.
- (a) Transformations return a new, modified RDD based on the original. Several transformations are available through spark API, including `map()`, `filter()`, `sample()`, and `union()`.
- (b) Actions return a value based on some computation being performed on an RDD. Some examples of actions supported by the spark API include `reduce()`, `count()`, `first()`, and `foreach()`.

Iterative and Interactive Applications

- Both **Iterative** and **Interactive** applications require faster data sharing across parallel jobs.
- Data sharing is slow in MapReduce due to **replication, serialization, and disk IO**.
- Regarding storage system, most of the Hadoop applications, they spend more than 90% of the time doing HDFS read-write operations.

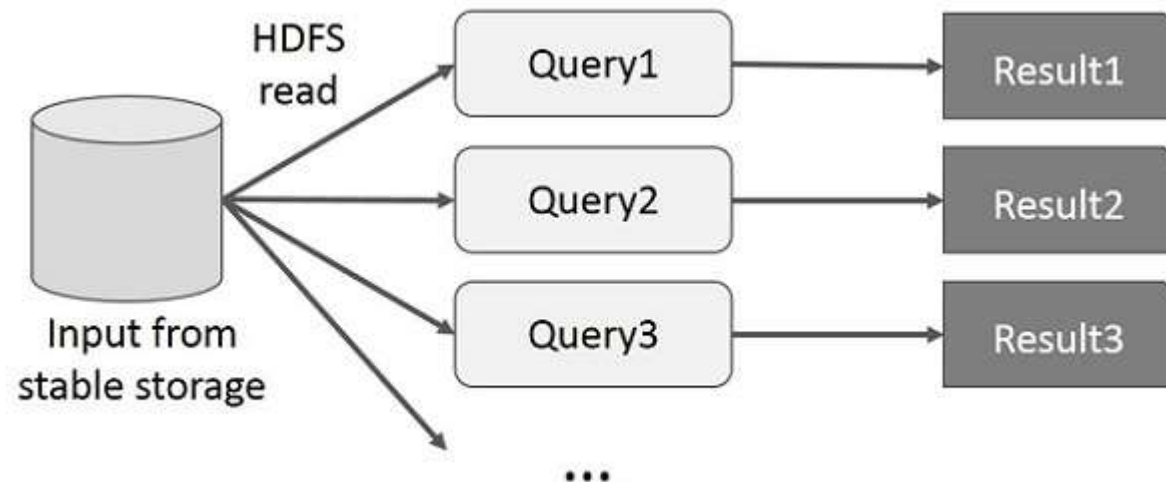
Iterative Operations on MapReduce

- Reuse intermediate results across multiple computations in multi-stage applications. The following illustration explains how the current framework works, while doing the iterative operations on MapReduce. This incurs substantial overheads due to data replication, disk I/O, and serialization, which makes the system slow.



Interactive Operations on MapReduce

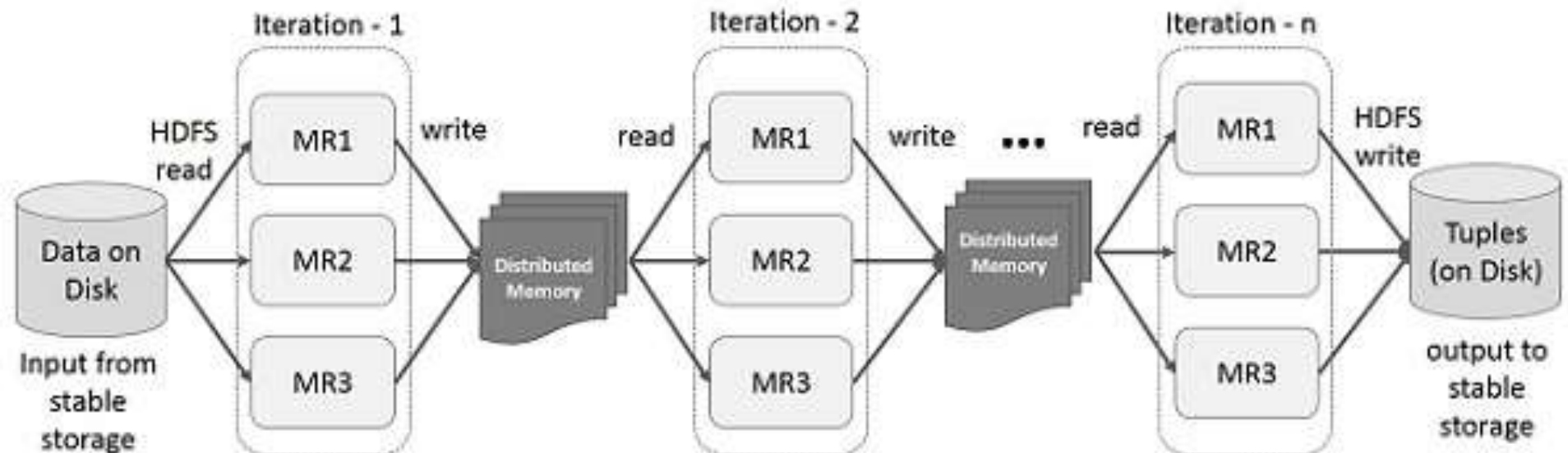
- User runs ad-hoc queries on the same subset of data. Each query will do the disk I/O on the stable storage, which can dominate application execution time.
- The following illustration explains how the current framework works while doing the interactive queries on MapReduce.



Iterative Operations on Spark RDD

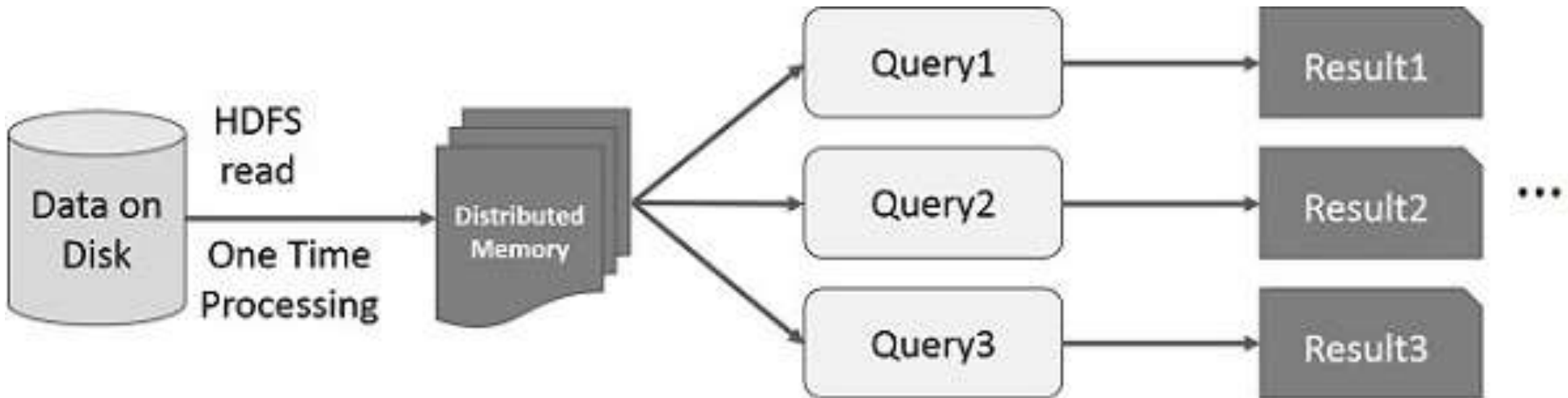
- The illustration given below shows the iterative operations on Spark RDD. It will store intermediate results in a distributed memory instead of Stable storage (Disk) and make the system faster.

Note – If the Distributed memory (RAM) is not sufficient to store intermediate results (State of the JOB), then it will store those results on the disk.



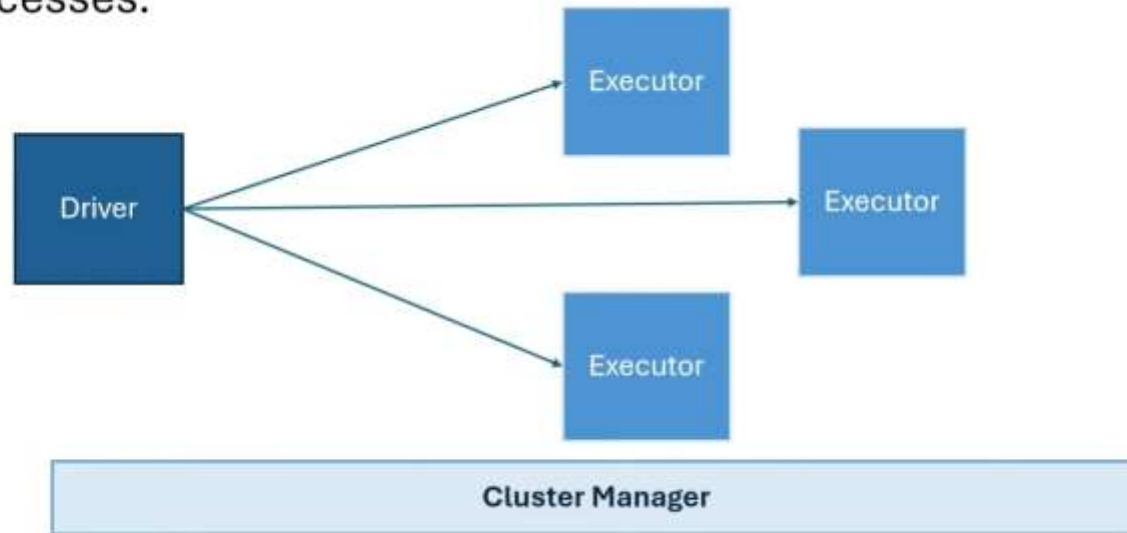
Interactive Operations on Spark RDD

- This illustration shows interactive operations on Spark RDD. If different queries are run on the same set of data repeatedly, this particular data can be kept in memory for better execution times.
- By default, each transformed RDD may be recomputed each time you run an action on it. However, you may also **persist** an RDD in memory, in which case Spark will keep the elements around on the cluster for much faster access, the next time you query it. There is also support for persisting RDDs on disk, or replicated across multiple nodes.



Basics

Spark Applications consist of a driver process and a set of executor processes.





Driver

- The driver process runs the `main()` function
- Sits on a node in the cluster
- Responsible for :
 - Maintaining information about the Spark Application
 - Responding to a user's program or input; and analyzing, distributing, and scheduling work across the executors

The driver process is **Essential**

it's the heart of a Spark Application and maintains all relevant information during the lifetime of the application.

Executor

- The executors are responsible for actually executing the work that the driver assigns them.
- Each executor is responsible for :
 - Executing code assigned to it by the driver
 - Reporting the state of the computation back to the driver node.



Cluster Manager

- The cluster manager controls physical machines and allocates resources to Spark Applications.
- Core cluster managers:
 - Spark's standalone cluster manager
 - YARN
 - Mesos

There can be multiple Spark Applications running on a cluster at the same time.



Spark session

The entry point into all functionality in Spark is the Spark Session class.

```
Import org.apache.spark.sql.SparkSession;
```

```
SparkSession spark=SparkSession.builder()  
.appName("Java Spark SQL basic example")  
.config("spark.some.config.option","some-value")  
.getOrCreate();
```

Dataframe

- A DataFrame is the most common Structured API
- Represents a table of data with rows and columns.
- The list of columns and the types in those columns the schema



Partition

- In order to allow every executor to perform work in parallel, Spark breaks up the data into chunks, called partitions.
- A partition is a collection of rows that sit on one physical machine in our cluster.
- A DataFrame's partitions represent how the data is physically distributed across the cluster of machines during execution.
- If we have one partition, Spark will only have a parallelism of one even if we have thousands of executors.
- If we have many partitions, but only one executor Spark will still only have a parallelism of one because there is only one computation resource.



Transformation

- RDD Transformations are Spark operations when executed on RDD, it results in a single or multiple new RDD's.
- Since RDD are immutable in nature, transformations always create new RDD without updating an existing one
- This creates an RDD lineage.

A dark blue square with a lighter blue circle in the center. The text "Lazy transformation" is written in white inside the circle.

Lazy transformation

RDD Transformations are lazy operations meaning none of the transformations get executed until an action on Spark RDD is called.



Narrow transformations

- Narrow transformations are the result of `map()` and `filter()` functions and these compute data that lives on a single partition and there will not be any data movement between partitions to execute narrow transformations.

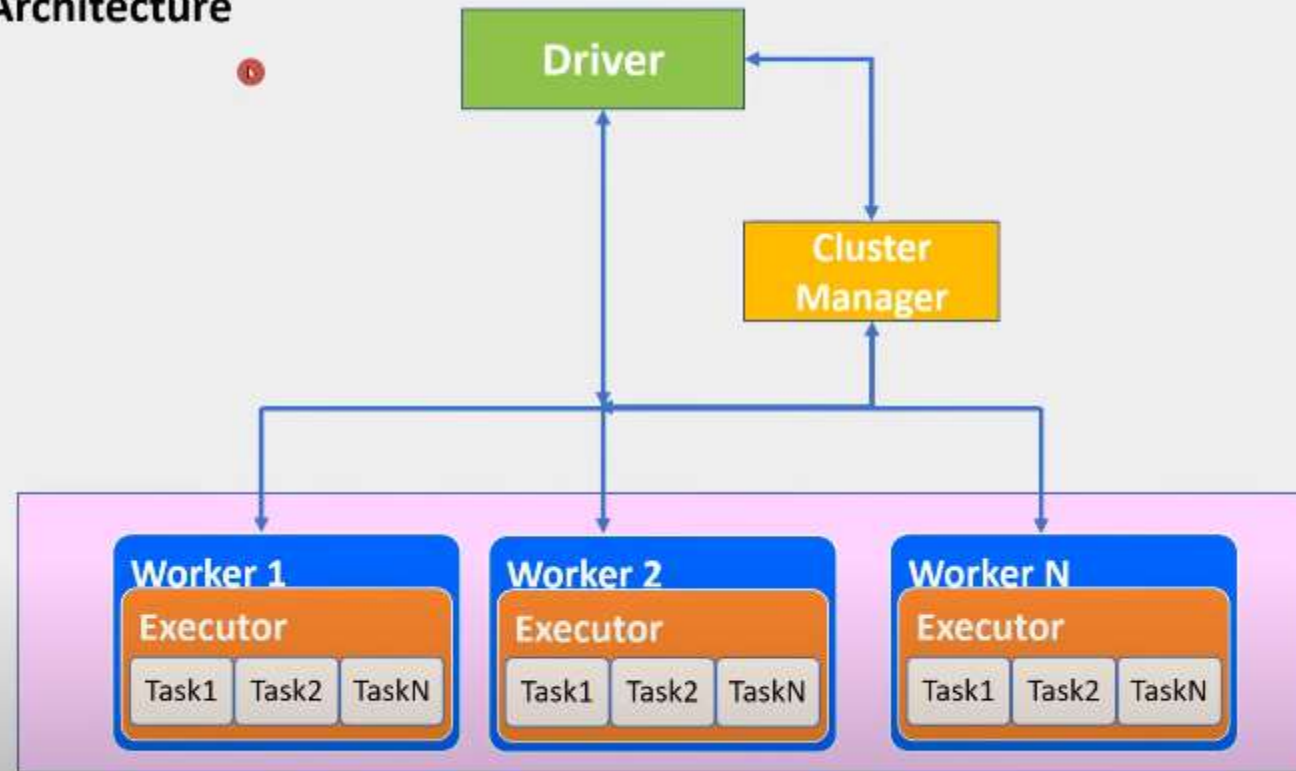




Wide transformations

- Wider transformations are the result of `groupByKey()` and `reduceByKey()` functions and these compute data that live on many partitions meaning there will be data movements between partitions to execute wider transformations. Since these shuffles the data, they also called shuffle transformations.
-

Architecture



RDD size = 2GB
Partition = 200
Partition size = 10MB
Partition/executor=50

Worker node: 64 core processor + 128 GB Memory

Executor 1: RAM = 32 GB

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16

Executor 2: RAM = 32 GB

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16

Executor 3: RAM = 32 GB

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16

Executor 4: RAM = 32 GB

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16

Lifecycle of Spark Application



- Step 1: User Submits Application
 - User runs: `spark-submit myApp.py`
 - Now Spark application starts.
- Step 2: Driver Initiates Spark Session
 - Driver: Starts JVM Creates SparkSession
 - Connects to cluster manager
 - System is now ready.
- Step 3: DAG Creates Logical Plan
 - Driver analyzes your transformations: `map` → `filter` → `reduce`
 - Creates: DAG (Directed Acyclic Graph) Still NO execution.
- Step 4: Task Executor Requests Resources
 - Driver asks cluster manager: "Give me executors"
- Step 5: Cluster Manager Allocates Resources
 - Cluster manager: Finds free workers
 - Launches executors
 - Allocates memory & CPU
 - Now cluster is ready.

- Step 6: Driver Establishes Connection with Workers
 - Driver: Sends tasks
 - Sends serialized functions
 - Sends metadata
 - Executors are now active.
- Step 7: Workers Execute Tasks
 - Executors: Read data from HDFS
 - Process partitions
 - Perform transformations
 - Do shuffle if needed
 - This is where real computation happens.
- Step 8: Workers Return Results to Driver
 - For actions like: `count()``collect()``take()`
 - Results go back to driver.
- Step 9: Driver Returns Result to User
 - User sees output.
- Step 10: Application Ends
 - Driver shuts down.
 - Executors removed.
 - Resources released.

- Use **repartition** when increasing partitions
- Use **coalesce** when decreasing partitions

Terminologies

- **Driver and worker Process:** These are nothing but JVM process. Within one worker node, there could be multiple executors. Each executor runs its own JVM process.
- **Application:** It could be single command or combination of multiple notebooks with complex logic. When code is submitted to spark for execution, Application starts.
- **Jobs:** When an application is submitted to Spark, driver process converts the code into job.
- **Stage:** Jobs are divided into stages. If the application code demands shuffling the data across nodes, new stage is created. Number of stages are determined by number of shuffling operations. Join is example of shuffling operation
- **Tasks:** Stages are further divided into multiple tasks. In a stage, all the tasks would execute same logic. Each task will process 1 partition at a time. So number of partition in the distributed cluster determines the number of tasks in each stage

Fundamentals

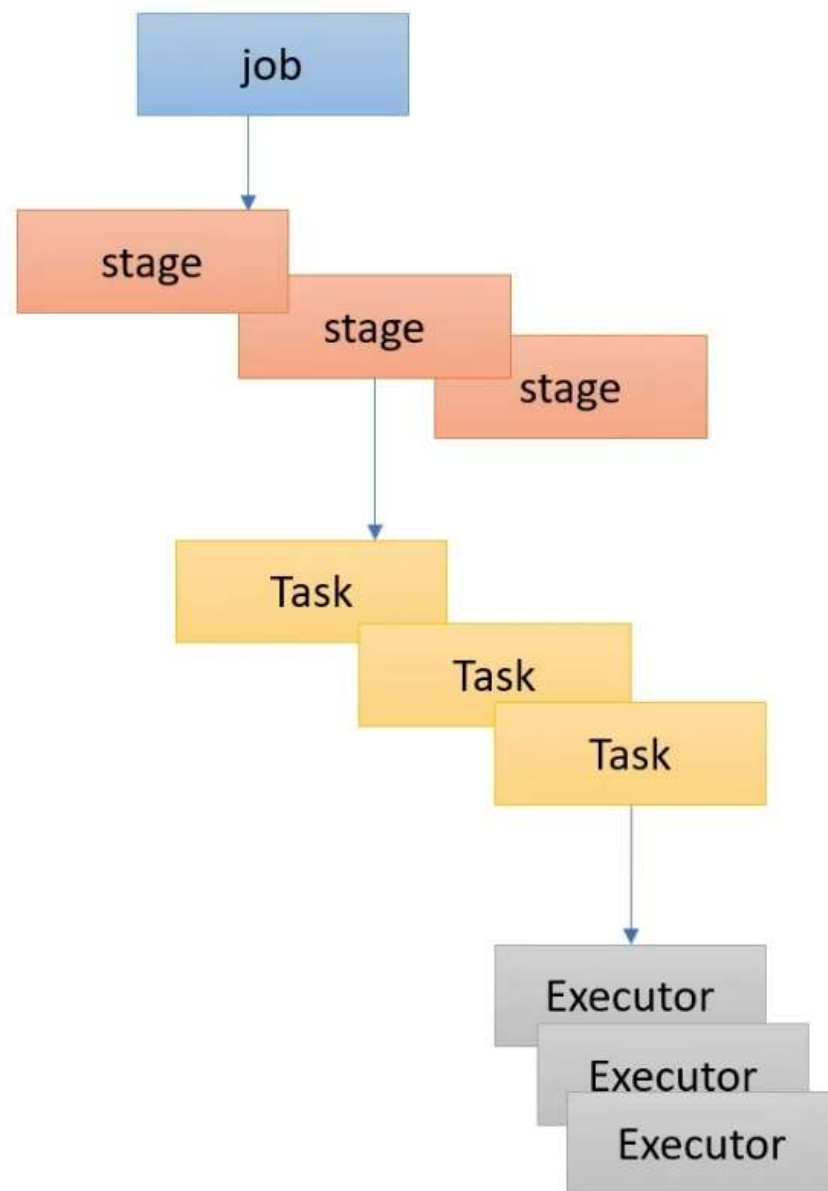
Job: A high-level computation triggered by an action (e.g., `collect()`, `count()`, or `save()`) on an RDD or DataFrame.

Stages: Each job is divided into stages based on transformations.

Tasks: Each stage consists of tasks that are executed in parallel on data partitions

Tasks are distributed to executors, which run on worker nodes in the cluster.

Executors perform the actual computation and store intermediate data.

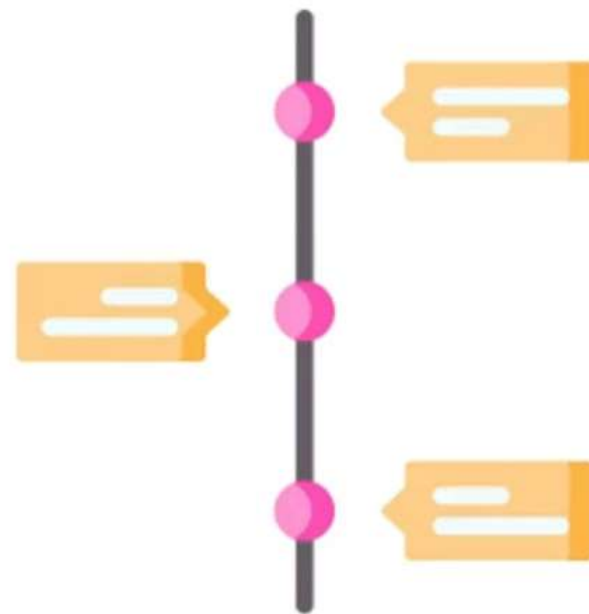


What are Stages in Spark?

A Stage is a group of operations that can be executed together without requiring data to be shuffled across the cluster.

Spark divides a DAG into multiple stages based on shuffle boundaries.

Each stage = set of tasks that can run in parallel



What are Tasks?

- A **Task** is the **smallest unit of execution** in Spark.
- Each **task** is a piece of a stage that runs on a **single partition** of data.
- If you have 200 partitions, Spark will run **200 tasks** in that stage.

A **Stage** is a *recipe*, and each **Task** is an *individual chef* cooking that part of the meal on a different burner.



Job Scheduling



Job scheduling in Apache Spark refers to the process of managing the execution of multiple jobs and tasks on a cluster.

It ensures efficient resource utilization, fairness, and optimization of computation across the distributed environment.

Job Scheduling Techniques



Fair Scheduling:

Ensures resources are distributed equally among jobs.
Each job gets a fair share of the cluster resources.
Configured using the fair scheduling mode.

FIFO (First-In-First-Out) Scheduling:

Jobs are scheduled in the order they are submitted.
This is the default scheduling mode in Spark.

Dynamic Resource Allocation:

Spark can dynamically adjust resources (e.g., add/remove executors) based on workload requirements.
Optimizes resource usage and improves cluster efficiency

Common Use Cases

1. Batch Processing:

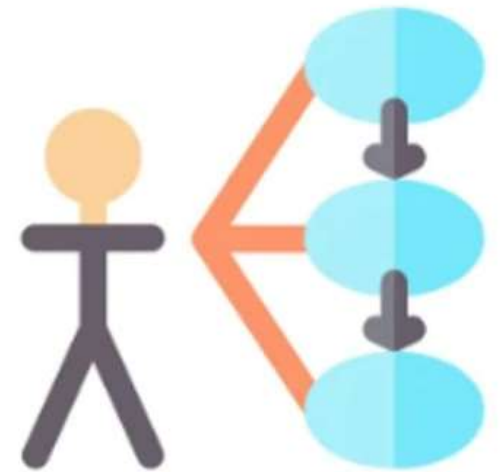
1. Jobs process large-scale static datasets.
2. Scheduling ensures efficient execution without overloading resources.

2. Streaming Applications:

1. Continuous processing of data streams.
2. Requires low-latency task scheduling.

3. Multi-Tenant Clusters:

1. Fair scheduling allows multiple users to run jobs without resource contention.

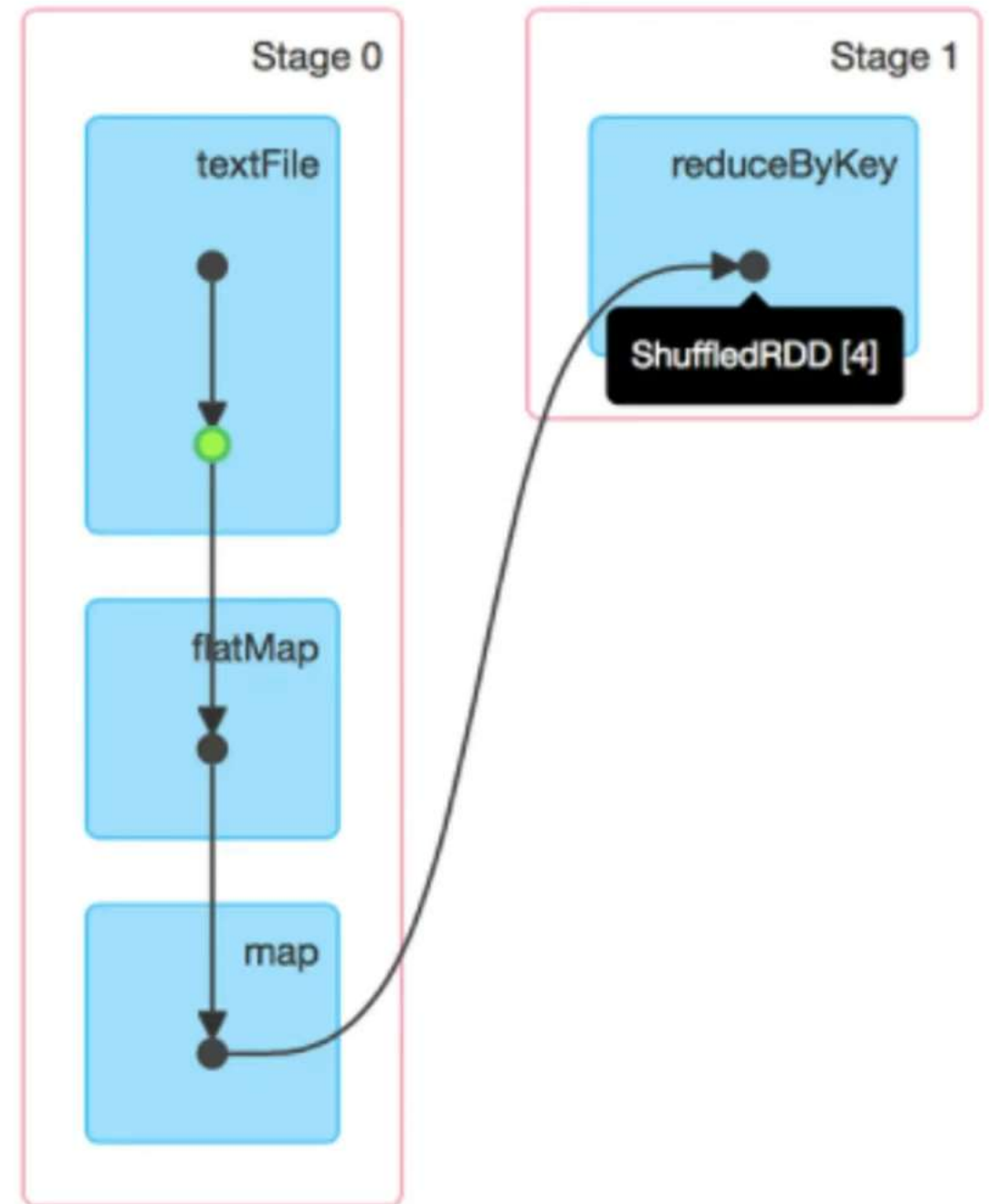


Spark Operations

Transformations (define a new RDD)	map filter sample groupByKey reduceByKey sortByKey	flatMap union join cogroup cross mapValues
Actions (return a result to driver program)	collect reduce count save lookupKey	

DAG Visualization

Blue box – Spark operation
Black Dots – RDDs created
Green Dot – Cached RDD in stage



- <https://www.databricks.com/blog/2015/06/22/understanding-your-spark-application-through-visualization.html>

Spark EcoSystem

- Spark is an integrated stack of tools responsible for scheduling, distributing, and monitoring applications consisting of many computational tasks across many workers machines, or a computing cluster.
- Spark is written primarily in Scala, but includes code from Python, Java, R and other languages.
- Spark comes with a set of integrated tools that reduces learning time and deliver higher user productivity, Spark ecosystem include Mesos resource manager, and other tools.
- Spark has already overtaken Hadoop in general because of benefits it provides in terms of faster execution in iterative processing algorithms.

SPARK Applications

- Real time log data monitoring
- Massive Natural language processing
- Large scale online recommendation systems
-

Spark vs Hadoop

Feature	Hadoop	Spark
Purpose	Resilient cost-effective storage and processing of large data sets	Fast general purpose engine for large scale data processing
Core component	Hadoop Distributed File System (HDFS)	Spark Core, the in-memory processing engine
Storage	HDFS manages massive data collections across multiple nodes within a cluster of commodity servers.	Spark doesn't do distributed storage. It operates on distributed data collections.
Fault tolerance	Hadoop uses replication to achieve fault tolerance	Spark uses RDD for fault tolerance that minimizes network I/O
Nature of processing	Accompanied by MapReduce, it includes batch processing of this data in parallel mode	Batch as well as stream processing
Sweet spot	Batch Processing	Iterative and interactive processing jobs that can fit in the memory.
Processing speed	Map Reduce is slow	Spark can be up to 10x faster than MR for batch processing and up to 100x faster for stream processing

Spark vs Hadoop

Feature	Hadoop	Spark
Security	More Secure	Less Secure
Failure recovery	Hadoop can recover from system faults or failures since data are written to disk after every operation	With Spark, data objects are stored in RDD, These can be reconstructed after faults or failures
Analytics tools	Separate engine	Built-in MLlib and GraphX libraries
Compatibility	Primary storage model is HDFS	Compatibility with HDFS and other storage formats
Language Support	Java	Scala is native language. APIs for python, R, Java, others
Driving organization	Yahoo	AMPLabs from UCBerkeley
Technology owners	Apache, Open-source, free	Open-source, free
Key distributors	Cloudera, Horton, MapR	Databricks, AMPLabs
Cost of System	Medium to High	Medium to High

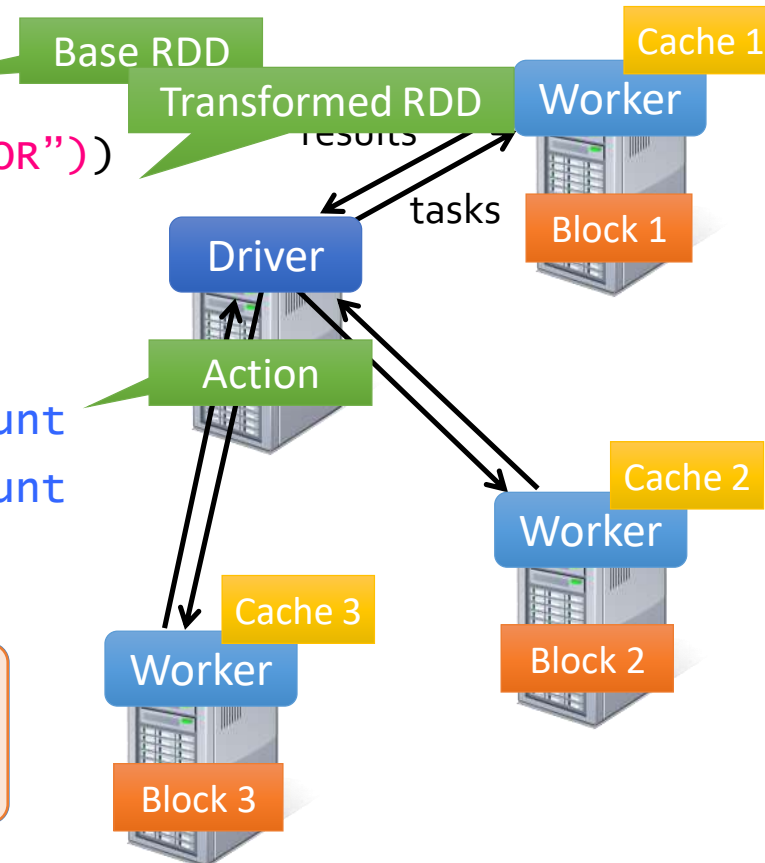
Example: Log Mining

Load error messages from a log into memory, then interactively search for various patterns

```
lines = spark.textFile("hdfs://...")
errors = lines.filter(_.startsWith("ERROR"))
messages = errors.map(_.split('\t')(2))
cachedMsgs = messages.cache()
```

```
cachedMsgs.filter(_.contains("foo")).count
cachedMsgs.filter(_.contains("bar")).count
. . .
```

Result: scaled to 1 TB data in 5-7 sec
(vs 170 sec for on-disk data)

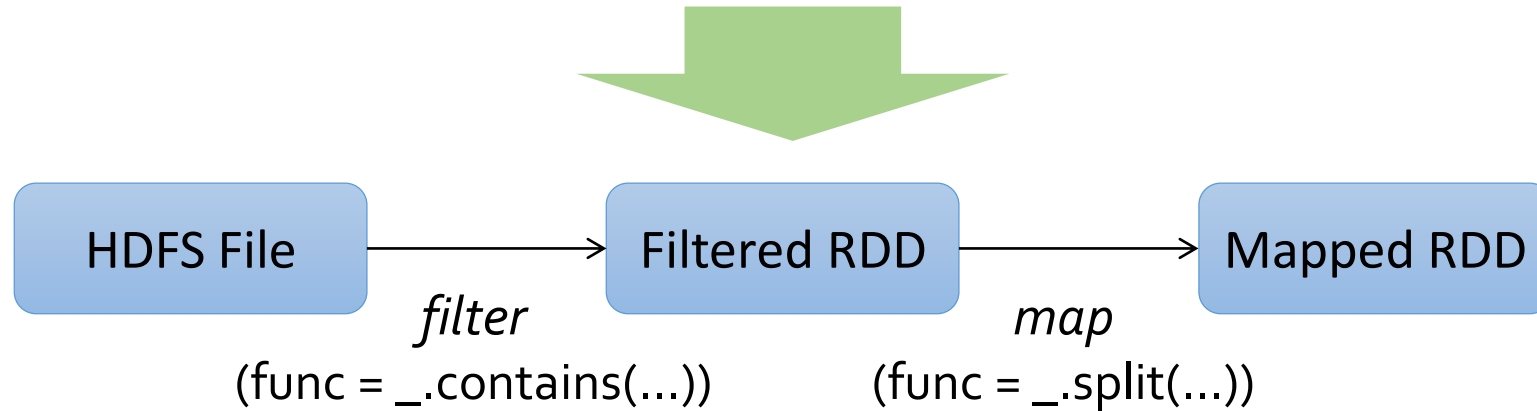


RDD Fault Tolerance

RDDs maintain *lineage* information that can be used to reconstruct lost partitions

Ex:

```
messages = textFile(...).filter(_.startsWith("ERROR"))  
                        .map(_.split('\t')(2))
```



Example: Logistic Regression

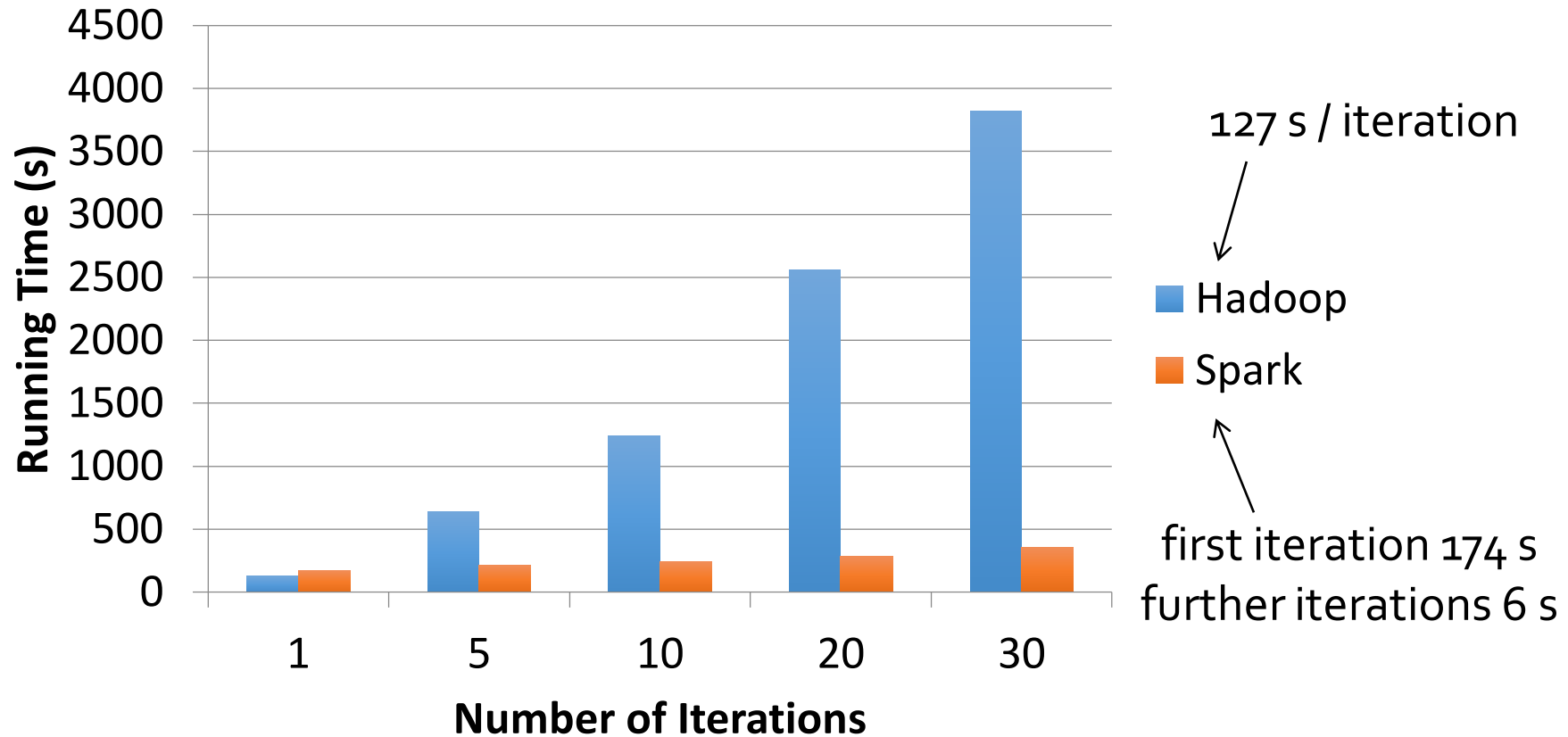
```
val data = spark.textFile(...).map(readPoint).cache()

var w = Vector.random(D)

for (i <- 1 to ITERATIONS) {
  val gradient = data.map(p =>
    (1 / (1 + exp(-p.y*(w dot p.x))) - 1) * p.y * p.x
  ).reduce(_ + _)
  w -= gradient
}

println("Final w: " + w)
```

Logistic Regression Performance



Spark Applications

- In-memory data mining on Hive data (Conviva)
- Predictive analytics (Quantifind)
- City traffic prediction (Mobile Millennium)
- Twitter spam classification (Monarch)

Collaborative filtering via matrix factorization

Events



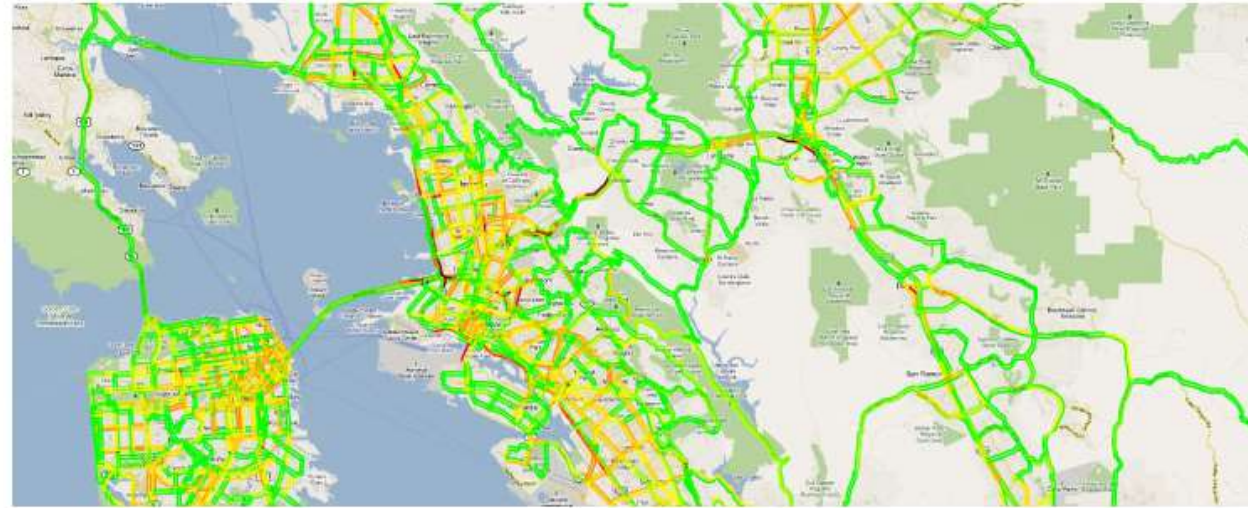
Mobile Millennium presented at the 2011 Transportation Research Board annual conference workshop on pervasive data, Washington DC, [see more](#)

Data Fusion



Major efforts on data fusion and data procurements now underway under Caltrans' leadership, [see more](#)

Snapshot of Mobile Millennium Traffic in San Francisco and the Bay Area



Project Overview

Mobile Millennium is a research project that includes a pilot traffic-monitoring system that uses the GPS in cellular phones to gather traffic information, process it, and distribute it back to the phones in real time.

The public-private research partnership — UC Berkeley, Nokia Research Center, and NAVTEQ, with sponsorship from the California Department of Transportation — launched the pilot program from the Berkeley campus on November 10, 2008. It ran for exactly 12 months. During that time, more than 5,000 users downloaded the Mobile Millennium traffic software onto their phones.

Frameworks Built on Spark

- Pregel on Spark (Bagel)
 - Google message passing model for graph computation
 - 200 lines of code
- Hive on Spark (Shark)
 - 3000 lines of code
 - Compatible with Apache Hive
 - ML operators in Scala

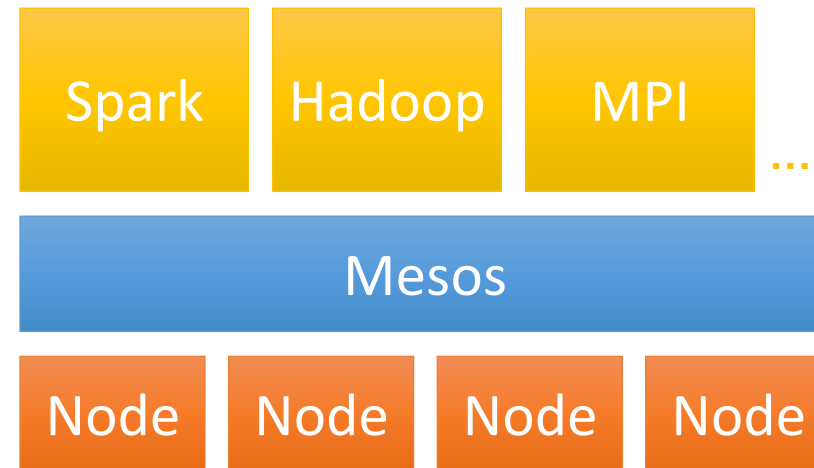


Implementation

Runs on Apache Mesos to share resources with Hadoop & other apps

Can read from any Hadoop input source (e.g. HDFS)

- No changes to Scala compiler

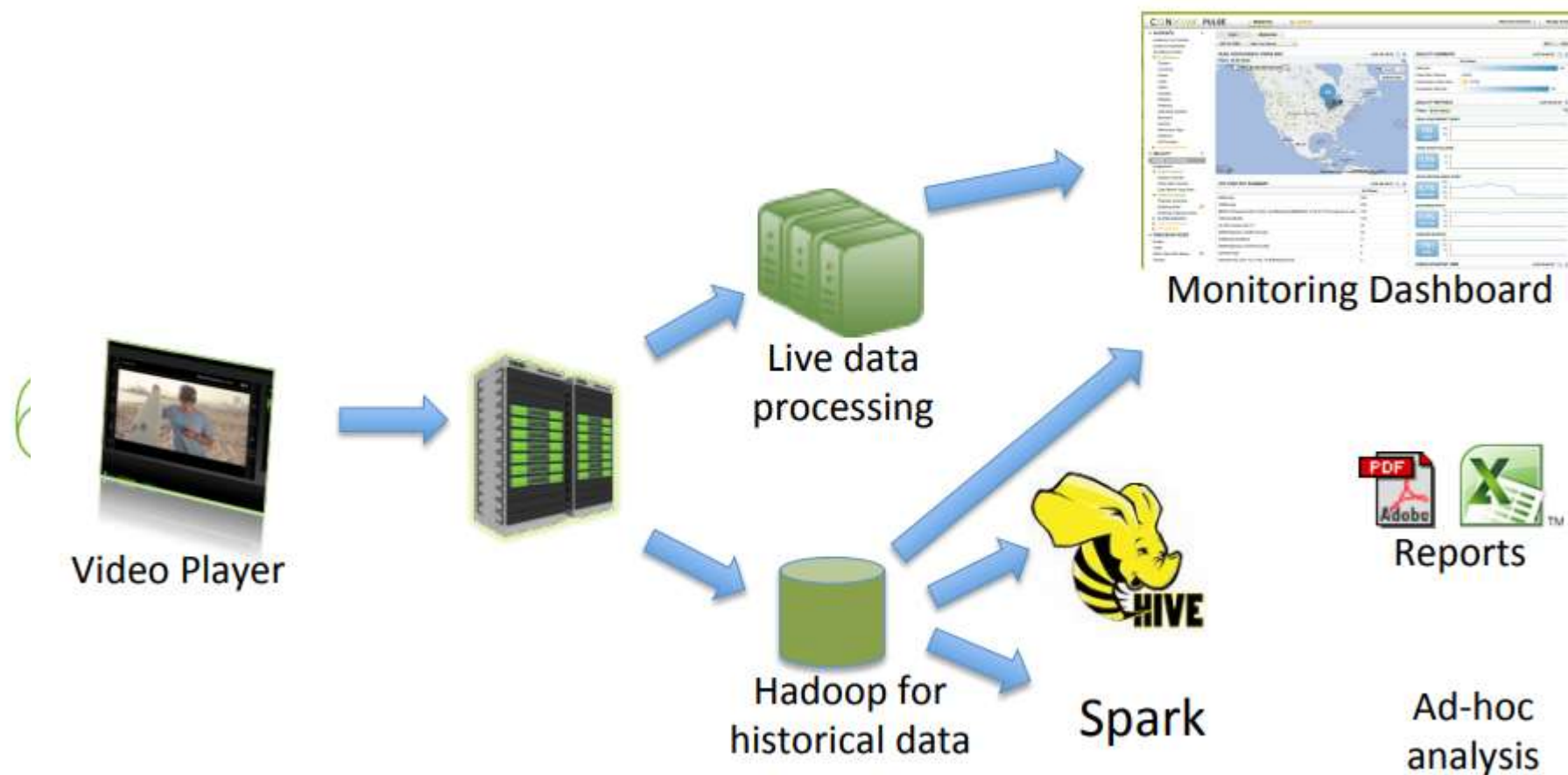


Case study: Conviva, Inc.

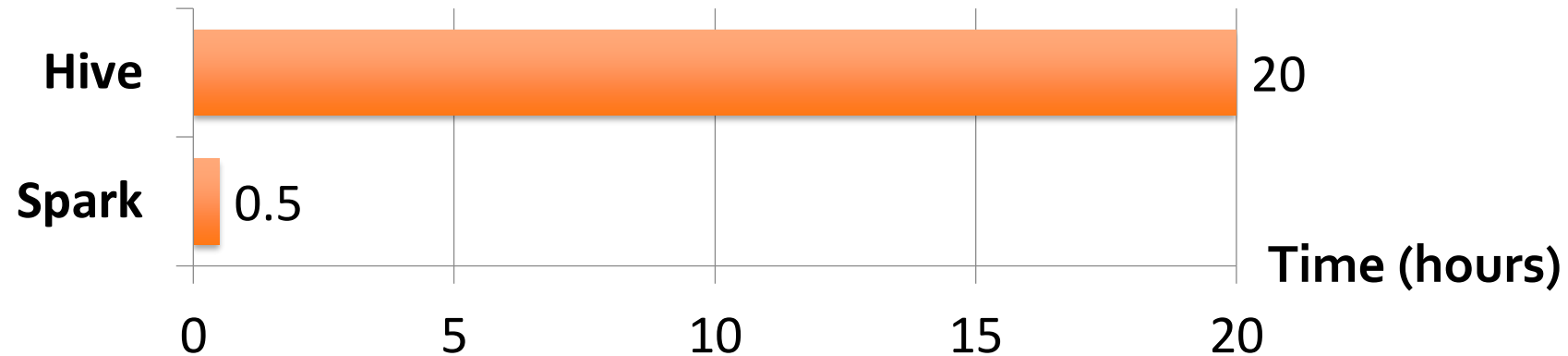
- Real-time monitoring of online video metadata
 - HBO, ESPN, ABC, SyFy, ...

- Two processing stacks
 - Custom-built distributed stream processing system
 - 1000s complex metrics on millions of video sessions
 - Requires many dozens of nodes for processing
 - Hadoop backend for offline analysis
 - Generating daily and monthly reports
 - **Similar computation as the streaming system**

Conviva data processing architecture



Conviva GeoReport



- Aggregations on many keys w/ same WHERE clause
- 40× gain comes from:
 - Not re-reading unused columns or filtered records
 - Avoiding repeated decompression
 - In-memory storage of objects

Case study: XYZ, Inc.

- Any company who wants to process live streaming data has this problem
- **Twice** the effort to implement any new function
- **Twice** the number of bugs to solve
- **Twice** the headache

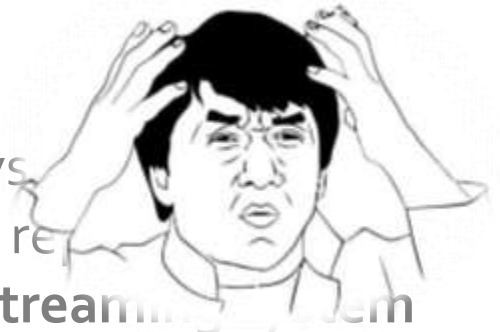
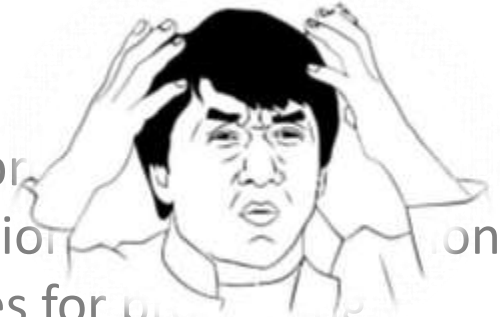
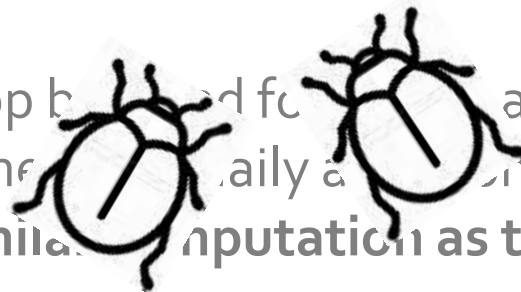
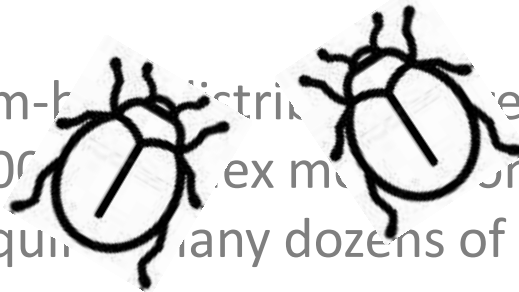
- Two processing stacks

Custom-built distributed stream processing

- 100% complex machine on million of nodes
- Requires many dozens of nodes for processing

Hadoop based for batch analysis

- Generally daily and monthly reports
- Similar computation as the streaming system

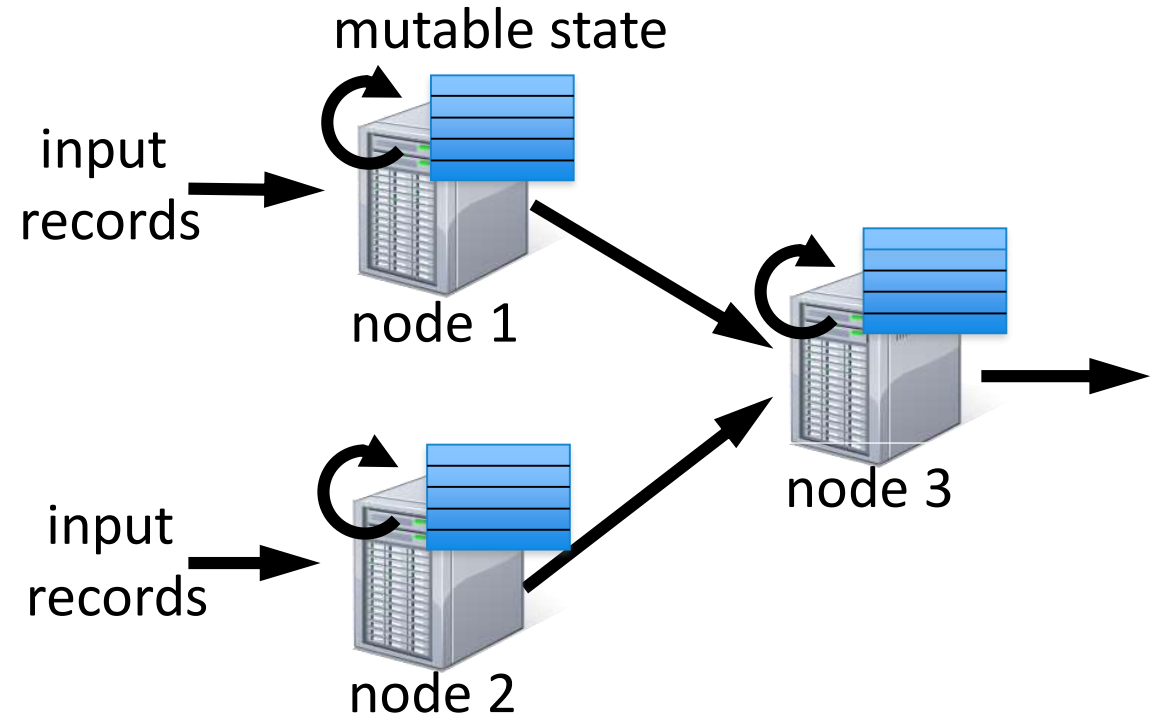


Requirements

- **Scalable** to large clusters
- **Second-scale** latencies
- **Simple** programming model
- **Integrated** with batch & interactive processing

Stateful Stream Processing

- Traditional streaming systems have a event-driven **record-at-a-time** processing model
 - Each node has mutable state
 - For each record, update state & send new records
- State is lost if node dies!
- Making stateful stream processing be fault-tolerant is challenging



Requirements

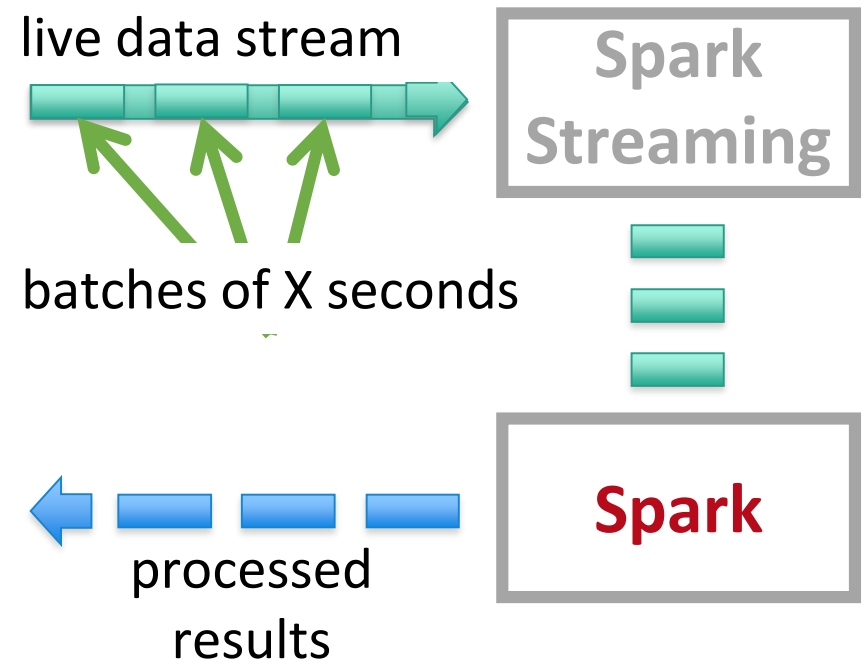
- **Scalable** to large clusters
- **Second-scale** latencies
- **Simple** programming model
- **Integrated** with batch & interactive processing
- **Efficient fault-tolerance** in stateful computations

Spark Streaming

Discretized Stream Processing

Run a streaming computation as a **series of very small, deterministic batch jobs**

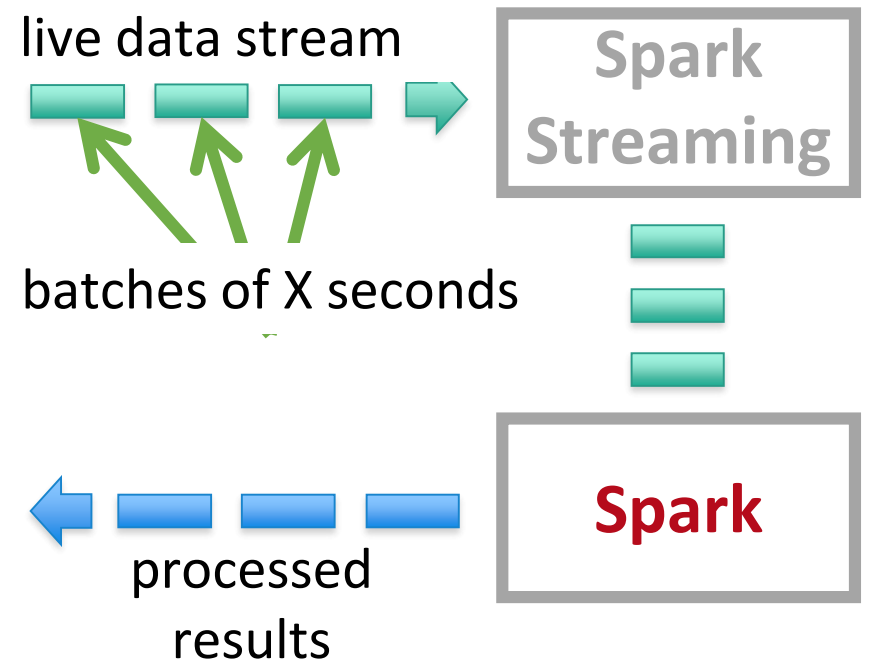
- Chop up the live stream into batches of X seconds
- Spark treats each batch of data as RDDs and processes them using RDD operations
- Finally, the processed results of the RDD operations are returned in batches



Discretized Stream Processing

Run a streaming computation as a **series of very small, deterministic batch jobs**

- Batch sizes as low as ½ second, latency ~ 1 second
- Potential for combining batch processing and streaming processing in the same system



Example 1 – Get hashtags from Twitter

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)
```

DStream: a sequence of RDD representing a stream of data

Twitter Streaming API

batch @ t

batch @ t+1

batch @ t+2



tweets DStream



stored in memory as an RDD
(immutable, distributed)

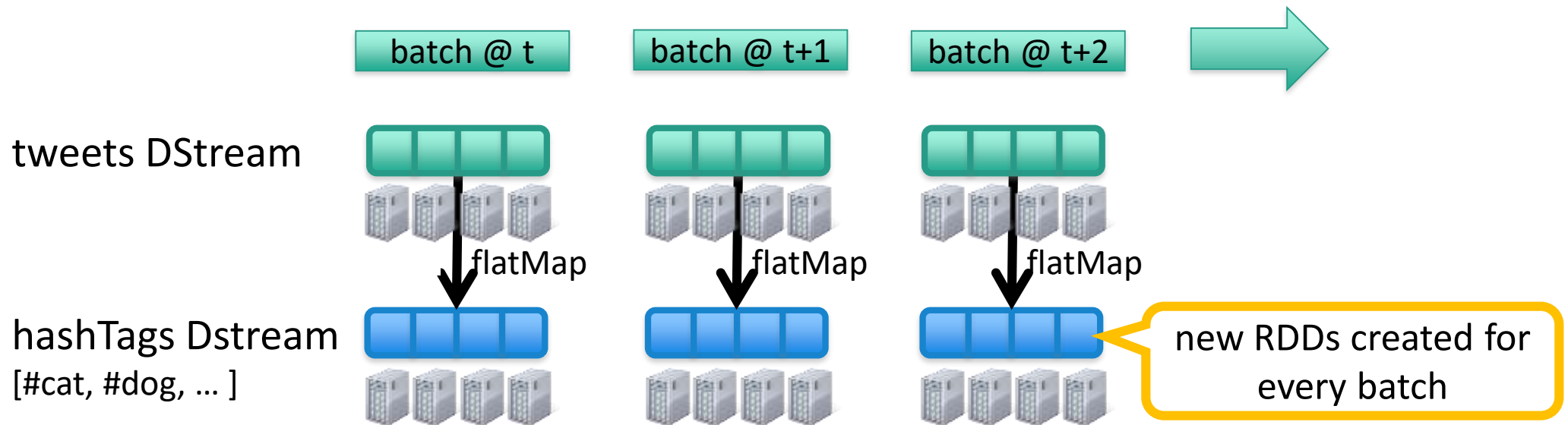
Example 1 – Get hashtags from Twitter

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)
```

```
val hashTags = tweets.flatMap(status => getTags(status))
```

new DStream

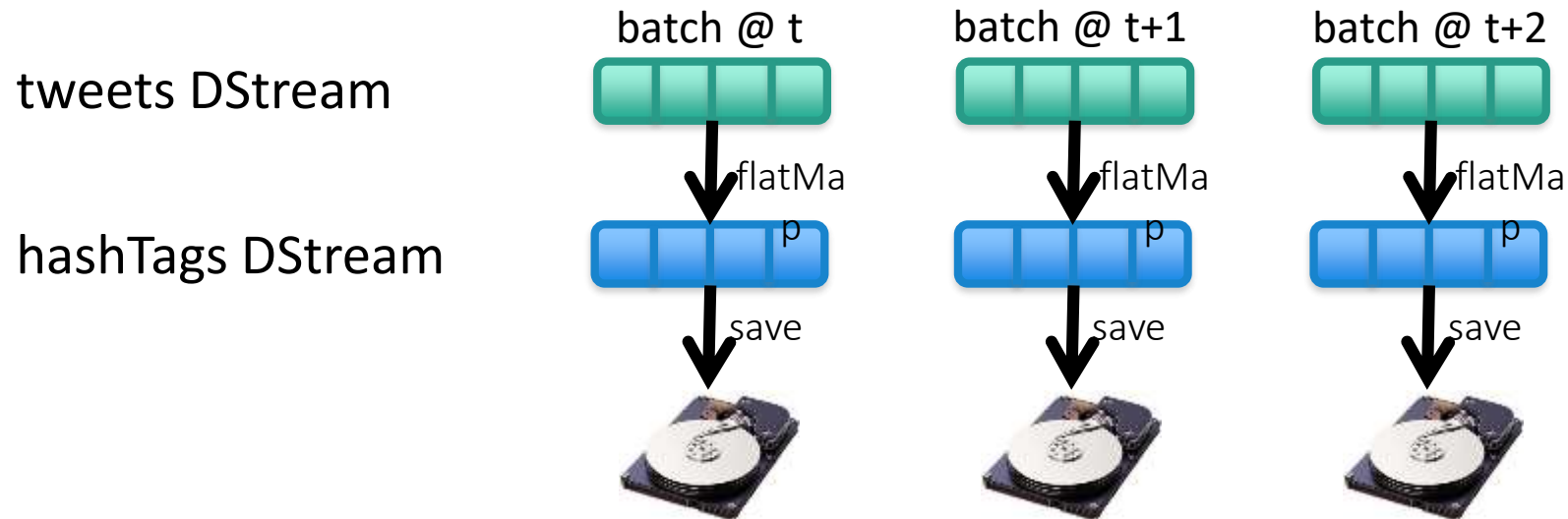
transformation: modify data in one Dstream to create another DStream



Example 1 – Get hashtags from Twitter

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)  
val hashTags = tweets.flatMap(status => getTags(status))  
hashTags.saveAsHadoopFiles("hdfs://...")
```

output operation: to push data to external storage



Java Example

Scala

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)  
val hashTags = tweets.flatMap(status => getTags(status))  
hashTags.saveAsHadoopFiles("hdfs://...")
```

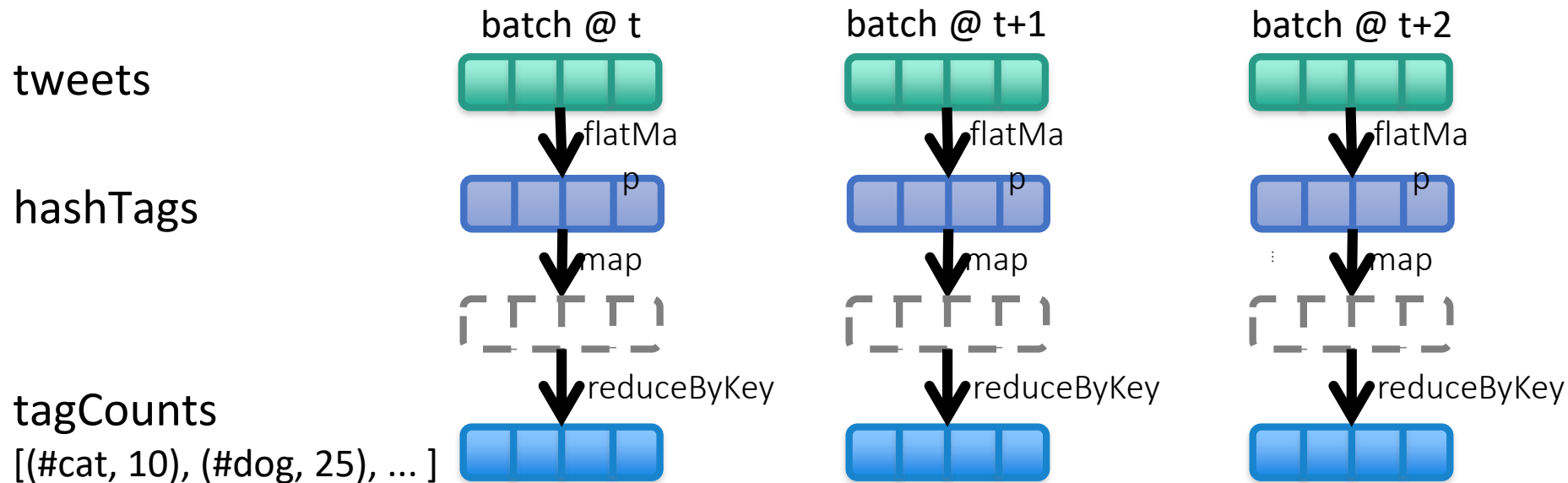
Java

```
JavaDStream<Status> tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)  
JavaDStream<String> hashTags = tweets.flatMap(new Function<...> {  })  
hashTags.saveAsHadoopFiles("hdfs://...")
```

Function object to define the transformation

Example 2 – Count the hashtags

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)  
val hashTags = tweets.flatMap(status => getTags(status))  
val tagCounts = hashTags.countByValue()
```



Example 3 – Count the hashtags over last 10 mins

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)  
val hashTags = tweets.flatMap (status => getTags(status))  
val tagCounts = hashTags.window(Minutes(10), Seconds(1)).countByValue()
```



sliding window
operation

window length

sliding interval